

CSIT213 Java Programming
Assignment 2 (15%)
Due: refer to Moodle

1. Objectives:

By the end of this assignment, students should be able to:

- To apply Object Oriented Design.
- To apply Object Oriented Programming.
- To implement solution in Java and IDE.

2. Overview

The National Fitness Council (NFC) oversees all registered fitness centres in the country. Each fitness centre offers a number of fitness classes (e.g. Yoga, Spin, HIIT), and members of the public sign up as participants of these classes. You are to build an object-oriented Java application that models this system.

You must implement five classes exactly as specified below, then verify your implementation using Java assertions (the `assert` statement, enabled with the `-ea` JVM flag).

The five classes are: `Person`, `Member`, `FitnessClass`, `FitnessCentre` and `FitnessCouncil`.

3. Class Specifications

3.1 Person

A base class representing any person in the system. All attributes are *private*.

Member / Method	Requirement
<code>String name</code>	The person's full name.
<code>String nric</code>	The person's national identification number (unique per person).
<code>String gender</code>	The person's gender.
<code>String dateOfBirth</code>	Date of birth, stored as a <code>String</code> (e.g. "1999-05-12").
<code>Person(name, nric, gender, dateOfBirth)</code>	Constructor initialising all four attributes.
<code>getName()</code> , <code>getNRIC()</code> , <code>getGender()</code> , <code>getDateOfBirth()</code>	Standard accessor (getter) methods for each attribute.
<code>String toString()</code>	Returns: name (nric), gender, DOB: dateOfBirth — e.g. "Alice Tan (S1234567A), Female, DOB: 1999-05-12"

3.2 Member (extends Person)

Represents a fitness centre member. `Member` must inherit from `Person` and reuse the superclass constructor via `super(...)`.

Member / Method	Requirement
<code>String membershipTier</code>	The member's tier, e.g. "Basic", "Premium" or "VIP".
<code>String preferredActivity</code>	The activity the member enjoys most, e.g. "Yoga".
<code>Member(name, nric, gender, dateOfBirth, membershipTier, preferredActivity)</code>	Constructor. Must call the superclass constructor for the inherited attributes.

Member / Method	Requirement
getMembershipTier(), getPreferredActivity()	Accessor methods for the two new attributes.
String toString()	Overrides Person's toString() and extends it (call super.toString()): name (nric), gender, DOB: dob, Tier: membershipTier, Prefers: preferredActivity

3.3 FitnessClass

Represents one scheduled fitness class run by a centre.

Member / Method	Requirement
String className	Name of the class, e.g. "Sunrise Yoga".
String activityType	Category of activity, e.g. "Yoga", "Spin", "HIIT".
int durationMinutes	Duration of one session in minutes.
ArrayList<Member> participants	Members enrolled in this class. Initially empty.
public static int maxParticipants = 2	Class-level (static) capacity limit shared by all classes. Default value 2 so that capacity is easy to test.
FitnessClass(className, activityType, durationMinutes)	Constructor initialising the three instance attributes.
boolean enrolMember(Member m)	Returns false if m is null, if a member with the same NRIC is already enrolled (no duplicates), or if the class is full (participants.size() >= maxParticipants). Otherwise adds the member and returns true.
boolean removeMember(String nric)	Returns false if nric is null or empty, or if no participant has that NRIC. NRIC matching must be case-insensitive. Removes the matching member and returns true.
getClassName(), getActivityType(), getParticipants()	Accessors. getParticipants() must never return null (return the empty list instead).
String toString()	Returns: className activityType durationMinutes mins X/Y enrolled where X is the current enrolment and Y is maxParticipants.

3.4 FitnessCentre

Represents one registered fitness centre. Classes are stored in a HashMap<String, ArrayList<FitnessClass>> where the key is the *activity type* and the value is the list of classes of that type offered by the centre.

Member / Method	Requirement
String centreName	The centre's name (unique across the council).
String registrationNumber	The centre's registration number, e.g. "FC-2026-001".
HashMap<String, ArrayList<FitnessClass>> classes	Key: activity type. Value: list of FitnessClass objects of that type. Initially empty.
FitnessCentre(centreName, registrationNumber)	Constructor.
boolean addClass(FitnessClass c)	Returns false if c is null, or if a class with the same className (case- insensitive) already exists under the same activity type in this centre. Otherwise inserts c into the list for its activity type (creating the list/key if needed) and returns true.

Member / Method	Requirement
boolean removeClass(String className)	Returns false if className is null/empty or not found. Match is case-insensitive across all activity types. On success, remove the class; if that activity type's list becomes empty, the key must also be removed from the HashMap. Returns true.
HashMap<String, ArrayList<FitnessClass>> getClasses()	Returns the map (never null; empty map if no classes).
ArrayList<FitnessClass> getClassesByActivity(String activityType)	Returns the list of classes for the given activity type, or an empty list (never null) if the centre has none.
getCentreName()	Accessor.
String toString()	Returns: centreName (Reg: registrationNumber)

3.5 FitnessCouncil

The top-level class. Maintains an ArrayList<FitnessCentre> of all registered centres.

Member / Method	Requirement
ArrayList<FitnessCentre> centres	All registered centres. Initially empty.
boolean registerCentre(FitnessCentre c)	Returns false if c is null or a centre with the same name (case-insensitive) is already registered. Otherwise adds it and returns true.
ArrayList<FitnessCentre> getCentres()	Returns the list of centres (never null).
ArrayList<String> getAllClassNames()	Returns the names of every class across every centre. Duplicate names across different centres must appear once per occurrence (no de-duplication).
ArrayList<String> getClassesByCentre(String centreName)	Returns the class names offered by the named centre (case-insensitive lookup). Returns an empty list if the centre does not exist or has no classes.
HashMap<String, ArrayList<FitnessClass>> getActivitiesByCentre(String centreName)	Returns the named centre's activity map (case-insensitive lookup). Returns an empty HashMap if the centre is not found.
ArrayList<String> getClassesByActivity(String activityType)	Returns the names of all classes of the given activity type across ALL centres, with duplicates removed (if two centres run a class with the same name, it appears once).

Skeleton codes for each classes are provided to assist you to kick start the Assignment.

Do spend some effort to attempt the assignment on your own.

Please remember:

- Your final exam is handwritten, closed-book, with no laptop, no IDE, and no AI assistance.
- That means your ability to write Java code from your own understanding is absolutely critical.
- You may score full marks for the assignment with external help, but if you score below 40 marks on the final exam, you will fail the module and will have to repeat it.
- Copying answers now might feel convenient, but it will not help you when it matters most.
- [You'll almost certainly be using AI in your future work, so it's equally important to be able to understand and evaluate AI-generated code, especially when it contains errors.](#)

Test Code and automated grading:

- The testing code `StudentAssertionTest.java` is provided to assist you to test your code. You need to enable assertion in Java to enable the testing.
- Ensure that you have passed all the test cases before submission, else zero marks will be awarded if the `StudentAssertionTest.java` failed to run.
- Another comprehensive code will be used to automate the grading of your assignment.
- Do not use any packages for the Java code as the automated grading will fail and will result in zero marks.

4. Submission

- You must adopt good programming practices which include (but not limited to) the followings:
 - Descriptive variable, parameter and method names.
 - Readable source code with proper indentation.
 - Descriptive block comments in the source code and the test results.
 - Modular programming.
 - Avoid global variables.
- You are encourage to develop more test cases in `StudentAssertionTest` to stress test your solution.
- **Do not use any packages for the Java code.**
- Failure to compile and execute the code will result in **zero marks awarded**.
- Please submit **one zip file** containing the following java classes: `Person`, `Member`, `FitnessClass`, `FitnessCentre` and `FitnessCouncil`
- Only zip file is accepted. Please do not use other compression format such as 7x, rar etc.
- For full time student, the zip file name must be in the form of: `TXXF_NAME_UOWID.zip` where XX is your tutorial group, NAME is your full name (without space and underscore) and UOW is your 7-digit UOW ID (**not** SIM ID).
For example: `T01F_JohnTan_9876543`.
- For part time student, the zip file name must be in the form of: `TXXP_NAME_UOWID.zip` where XX is your tutorial group, NAME is your full name (without space and underscore) and UOWID is your 7-digit UOW ID (**not** SIM ID).
For example: `T01P_JohnTan_9876543`.
- Wrong file naming (even an extra space, 5 marks will be deducted)
- Please ensure the UOWID is correct as it will be used by the marking scripts to update your assignment mark in Moodle.
- Automated scripts will be used to extract your files for grading. Failure to follow the above instructions will result in failure to extract and grade your submission.
- No appeal will be granted for incorrect submission such as empty zip file, wrong files, etc.
- Late submission will be penalized 5% per day late.
- Late submission after 4 days will not be graded.